Chapter 6: Functions

By Irina Galata

Functions are a core part of many programming languages. Simply put, a function lets you define a block of code that performs a task. Then, whenever your app needs to execute that task, you can run the function instead of having to copy and paste the same code everywhere.

In this chapter, you'll learn how to write your own functions, and see firsthand how Kotlin makes them easy to use.

Function basics

Imagine you have an app that frequently needs to print your name. You can write a function to do this. Try out this code:

```
fun printMyName() {
  println("My name is Joe Howard.")
}
```

The code above is known as a **function declaration**. You define a function using the `fun` keyword. After that comes the name of the function, followed by parentheses. You'll learn more about the need for these parentheses in the next section.

After the parentheses comes an opening brace, followed by the code you want to run in the function, followed by a closing brace. With your function defined, you can use it like so:

```
printMyName()
```

This prints out the following:

```
My name is Joe Howard.
```

If you suspect that you've already used a function in previous chapters, you're correct! `println`, which prints the text you give it to the console, is indeed a function. This leads nicely into the next section, in which you'll learn how to pass data to a function and get data back in return.

Function parameters

In the previous example, the function simply prints out a message. That's great, but sometimes you want to **parameterize** your function, which lets the function perform differently depending on the data passed into it via its **parameters**.

As an example, consider the following function:

```
fun printMultipleOfFive(value: Int) {
  println("$value * 5 = ${value * 5}")
}
printMultipleOfFive(10)
```

Here, you can see the definition of one parameter inside the parentheses after the function name, named `value` and of type `Int`. In any function, the parentheses contain what's known as the **parameter list**. These parentheses are required both when declaring and when invoking the function, even if the parameter list is empty. This function will print out any given multiple of five. In the example, you call the function with an **argument** of 10, so the function prints the following:

```
10 * 5 = 50
```

> **Note:** Take care not to confuse the terms "parameter" and "argument". A function declares its *parameters* in its parameter list. When you call a function, you provide values as *arguments* for the functions parameters.

You can take this one step further and make the function more general. With two parameters, the function can print out a multiple of any two values.

```
fun printMultipleOf(multiplier: Int, andValue: Int) {
  println("$multiplier * $andValue = ${multiplier * andValue}")
}
printMultipleOf(4, 2)
```

There are now two parameters inside the parentheses after the function name: one named `multiplier` and the other named `andValue`, both of type `Int`.

Parameter named arguments

Sometimes it is helpful to use **named arguments** when calling a function to make it easier to understand the purpose of each argument.

```
printMultipleOf(multiplier = 4, andValue = 2)
```

It is now immediately obvious at the *call site* of the function what purpose the arguments serve. This is especially helpful when a function has several parameters.

Parameter default values

You can also give **default values** to parameters:

```
fun printMultipleOf(multiplier: Int, value: Int = 1) {
  println("$multiplier * $value = ${multiplier * value}")
}

printMultipleOf(4)
```

The difference is the `= 1` after the second parameter, which means that if no value is provided for the second parameter, it defaults to 1.

Therefore, this code prints the following:

```
4 * 1 = 4
```

It can be useful to have a default value when you expect a parameter to be one particular value the majority of the time, and it will simplify your code when you call the function.

Return values

All of the functions you've seen so far have performed a simple task: Printing something out. Functions can also return a value. The caller of the function can assign the return value to a variable or constant, or use it directly in an expression.

This means you can use a function to manipulate data. You simply take in data through parameters, manipulate it and then return it.

Here's how you define a function that returns a value:

```
fun multiply(number: Int, multiplier: Int): Int {
  return number * multiplier
}
```

To declare that a function returns a value, you add a `:` followed by the type of the return value after the set of parentheses and before the opening brace. In this example, the function returns an `Int`.

Inside the function, you use a `return` statement to return the value. In this example, you return the product of the two parameters.

It's also possible to return multiple values through the use of `Pairs`:

```
fun multiplyAndDivide(number: Int, factor: Int): Pair<Int, Int>
{
  return Pair(number * factor, number / factor)
}
val (product, quotient) = multiplyAndDivide(4, 2)
```

This function returns *both* the product and quotient of the two parameters by returning a `Pair` containing two `Int` values.

If a function consists solely of a single expression, you can assign the expression to the function using = while at the same time not using braces, a return type, or a `return` statement:

```
fun multiplyInferred(number: Int, multiplier: Int) = number *
multiplier
```

In such a case, the type of the function return value is *inferred* to be the type of the expression assigned to the function. In the example, the return type is inferred to be `Int` since both `number` and `multiplier` are `Int`s.

Parameters as values

Function parameters are constants by default, which means they can't be modified.

To illustrate this point, consider the following code:

```
fun incrementAndPrint(value: Int) {
  value += 1
  print(value)
}
```

This results in an error:

```
Val cannot be reassigned
```

The parameter `value` is the equivalent of a constant declared with `val` and hence cannot be reassigned. Therefore, when the function attempts to increment it, the compiler emits an error.

Usually you want this behavior. Ideally, a function doesn't alter its parameters. If it did, then you couldn't be sure of the parameters' values and you might make incorrect assumptions in your code, leading to the wrong data.

If you want a function to alter a parameter and return it, you must do so indirectly by declaring a new variable like so:

```
fun incrementAndPrint(value: Int): Int {
  val newValue = value + 1
  println(newValue)
  return newValue
}
```

> **Note:** As you'll see in a later chapter, when adding parameters to the primary constructor when defining a class, you *do* add `var` or `val` to the parameters. You do this to indicate that the parameters are properties of the class and also that they either can or cannot be reassigned.

Overloading

What if you want more than one function with the same name?

```
fun getValue(value: Int): Int {
  return value + 1
}

fun getValue(value: String): String {
  return "The value is $value"
}
```

This is called **overloading** and lets you define similar functions using a single name.

However, the compiler must still be able to tell the difference between these functions within a given scope. Whenever you call a function, it should always be clear which function you're calling.

This is usually achieved through a difference in the parameter list:

- A different number of parameters.
- Different parameter types.

> **Note**: The return type alone is not enough to distinguish two functions.

For example, defining two methods like so will result in an error:

```
fun getValue(value: String): String {
  return "The value is $value"
}

// Conflicting overloads error
fun getValue(value: String): Int {
    return value.length
}
```

The methods above both have the same name, parameter types and number of parameters. Kotlin will not be able to distinguish them!

It's worth noting that overloading should be used with care. Only use overloading for functions that are related and similar in behavior.

Mini-exercises

1. Write a function named `printFullName` that takes two strings called `firstName` and `lastName`. The function should print out the full name defined as `firstName + " " + lastName`. Use it to print out your own full name.
2. Call `printFullName` using named arguments.
3. Write a function named `calculateFullName` that returns the full name as a string. Use it to store your own full name in a constant.
4. Change `calculateFullName` to return a `Pair` containing both the full name and the length of the name. You can find a string's length by using the `length` property. Use this function to determine the length of your own full name.

Functions as variables

Functions in Kotlin are simply another data type. You can assign them to variables and constants just as you can any other type of value, such as an `Int` or a `String`.

To see how this works, consider the following function:

```
fun add(a: Int, b: Int): Int {
  return a + b
}
```

This function takes two parameters and returns the sum of their values.

You can assign this function to a variable using the **method reference operator**, `::`, like so:

```
var function = ::add
```

Here, the name of the variable is `function` and its type is inferred as `(Int, Int) -> Int` from the `add` function you assigned to it. The `function` variable is of a function type that takes two `Int` parameters and returns an `Int`.

Now you can use the `function` variable in just the same way you'd use `add`, like so:

```
function(4, 2)
```

This returns 6.

Now consider the following code:

```
fun subtract(a: Int, b: Int) : Int {
  return a - b
}
```

Here, you declare another function that takes two `Int` parameters and returns an `Int`. You can set the `function` variable from before to your new `subtract` function, because the parameter list and return type of `subtract` are compatible with the type of the `function` variable.

```
function = ::subtract
function(4, 2)
```

This time, the call to `function` returns 2.

The fact that you can assign functions to variables comes in handy because it means you can pass functions to other functions. Here's an example of this in action:

```
fun printResult(function: (Int, Int) -> Int, a: Int, b: Int) {
  val result = function(a, b)
  print(result)
}
printResult(::add, 4, 2)
```

`printResult` takes three parameters:

1. `function` is of a function type that takes two `Int` parameters and returns an `Int`, declared like so: `(Int, Int) -> Int`.
2. `a` is of type `Int`.
3. `b` is of type `Int`.

`printResult` calls the passed-in function, passing into it the two `Int` parameters. Then it prints the result to the console:

```
6
```

It's extremely useful to be able to pass functions to other functions, and it can help you write reusable code. Not only can you pass data around to manipulate, but passing functions as parameters also means you can be flexible about what code executes.

Assigning functions to variables and passing functions around as arguments is one aspect of **functional programming**, which you'll learn much more about in Chapter 21.

The land of no return

There are some functions which are designed to never, ever, return. This may sound confusing, but consider the example of a function that is designed to crash an application. This may sound strange, but if an application is about to work with corrupt data, it's often best to crash rather than continue in an unknown and potentially dangerous state.

Another example of a non-returning function is one which handles an event loop. An event loop is at the heart of every modern application which takes input from the user and displays things on a screen. The event loop services requests coming from the user, then passes these events to the application code, which in turn causes the information to be displayed on the screen. The loop then cycles back and services the next event.

These event loops are often started in an application by calling a function which is known to never return. If you start developing Android apps, think back to this paragraph when you encounter the **main thread**, also known as the **UI thread**.

Kotlin has a way to tell the compiler that a function is known to never return. You set the return type of the function to the `Nothing` type, indicating that nothing is ever returned from the function.

A crude, but honest, implementation of a function that wouldn't return would be as follows:

```
fun infiniteLoop(): Nothing {
  while (true) {

  }
}
```

You may be wondering why bother with this special return type. It's useful because by the compiler knowing that the function won't ever return, it can make certain optimizations when generating the code to call the function.

Essentially, the code which calls the function doesn't need to bother doing anything after the function call, because it knows that this function will never end before the application is terminated.

Writing good functions

There are many ways to solve problems with functions. The best (easiest to use and understand) functions do *one simple task* rather than trying to do many. This makes them easier to mix and match and assemble into more complex behaviors. Good functions also have a well defined set of inputs that produce the same output every time. This makes them easier to reason about and test in isolation. Keep these rules-of-thumb in mind as you create functions.

Before you move on, check out the challenges ahead as you'll need to fully grasp functions before understanding some of the upcoming topics! If you need help when solving them, use the solutions included in the chapter materials for a hint.

Challenges

Challenge 1: It's prime time

When acquainting yourself with a programming language, a good exercise to do is write a function to determine whether or not a number is prime. That's your first challenge.

First, write the following function:

```
fun isNumberDivisible(number: Int, divisor: Int): Boolean
```

You'll use this to determine if one number is divisible by another. It should return `true` when `number` is divisible by `divisor`.

Hint: You can use the modulo (`%`) operator to help you out here.

Next, write the main function:

```
fun isPrime(number: Int): Boolean
```

This should return `true` if `number` is prime, and `false` otherwise. A number is prime if it's only divisible by 1 and itself. You should loop through the numbers from 1 to the number and find the number's divisors.

If it has any divisors other than 1 and itself, then the number isn't prime. You'll need to use the `isNumberDivisible()` function you wrote earlier.

Use this function to check the following cases:

```
isPrime(6) // false
isPrime(13) // true
isPrime(8893) // true
```

Hint 1: Numbers less than 0 should not be considered prime. Check for this case at the start of the function and return early if the number is less than 0.

Hint 2: Use a `for` loop to find divisors. If you start at 2 and end before the number itself, then as soon as you find a divisor, you can return `false`.

Hint 3: If you want to get *really* clever, you can simply loop from 2 until you reach the square root of `number`, rather than going all the way up to `number` itself. I'll leave it as an exercise for you to figure out why. It may help to think of the number 16, whose square root is 4. The divisors of 16 are 1, 2, 4, 8 and 16.

Challenge 2: Recursive functions

In this challenge, you're going to see what happens when a function calls *itself*, a behavior called **recursion**. This may sound unusual, but it can be quite useful.

You're going to write a function that computes a value from the **Fibonacci sequence**. Any value in the sequence is the sum of the previous two values. The sequence is defined such that the first two values equal 1. That is, `fibonacci(1) = 1` and `fibonacci(2) = 1`.

Write your function using the following declaration:

```
fun fibonacci(number: Int): Int
```

Then, verify you've written the function correctly by executing it with the following numbers:

```
fibonacci(1)  // = 1
fibonacci(2)  // = 1
fibonacci(3)  // = 2
fibonacci(4)  // = 3
fibonacci(5)  // = 5
fibonacci(6)  // = 8
fibonacci(7)  // = 13
fibonacci(10) // = 55
```

Hint 1: For values of `number` less than 0, you should return 0.

Hint 2: To start the sequence, hard-code a return value of 1 when `number` equals 1 or 2.

Hint 3: For any other value, you'll need to return the sum of calling `fibonacci` with `number - 1` and `number - 2`.

> **Note**: This way of calculating the Fibonacci numbers is not terribly efficient. One technique to improve the performance is called **memoization**, which stores the results of previous calculations and reuses them when possible.

Key points

- You use a **function** to define a task, which you can execute as many times as you like without having to write the code multiple times.
- Functions can take zero or more **parameters** and optionally return a value.
- For clarity at the call site you can use named arguments when calling a function.
- Specifying default function values can make those functions easier to work with and reduce the amount of code you have to write.
- Functions can have the same name with different parameters. This is called **overloading**.
- You can assign functions to variables and pass them to other functions.
- Functions can have a special `Nothing` return type to inform Kotlin that this function will never exit.
- Strive to create functions that are clearly named and have one job with repeatable inputs and outputs.

Where to go from here?

Functions are the first step in grouping small pieces of code together into a larger unit. In the next chapter, you'll learn about **nullability**, which is an important part of Kotlin's syntactic arsenal.